CSC 213 - Programming Language (2)

DR. ABDULLAH ALBALAWI

Course outlines

- Topic 1: Object Oriented Programming
- Topic 2: Exception Handling
- Topic 3: Regular expressions
- Topic 4: File Handling
- Topic 5: Database
- Topic 6: Multithreading
- Topic 7: GUI Programming
- Topic 8: Data Visualization in Python

Course Info

Software Tools

- Python 3.11
  - Python Interpreter and IDLE IDE.
  - Download: https://www.python.org/downloads/

Course Info

- Grading

Midterm Exam (1)	20pt	60pt
Midterm Exam (2)	20pt	
Quizzes, Assignments	10pt	
Project, lab exam	10pt	
Final exam	40pt	
Total points	100pt	

Contact

Email: aalbalawi@su.edu.sa

Object Oriented Programming

Lecture 1

Introduction to Object-Oriented Programming (OOP)

- Object-Oriented Programming (OOP) is a programming paradigm that structures code around the concept of "objects."
  - These objects encapsulate data and behavior, making it easier to model and interact with real-world entities.
  - Object oriented programming is a way to think about “objects” in a program (such as variables, functions, etc)
  - A program becomes less a list of instruction and more a set of objects and how they interact.
  - Everything in Python is an object.

Introduction to Object-Oriented Programming (OOP)

- OOP is based on four main principles: encapsulation, modularity, inheritance, and polymorphism.
  - **Encapsulation:** hiding design details to make the program clearer and more easily modified later.
  - **Modularity:** the ability to make objects stand alone so they can be reused (our modules). Like the math module.
  - **Inheritance:** create a new object by inheriting (like father to son) many object characteristics while creating or over-riding for this object.
  - **Polymorphism**: allow one message to be sent to any object and have it respond appropriately based on the type of object it is.

OOP helps for software engineering

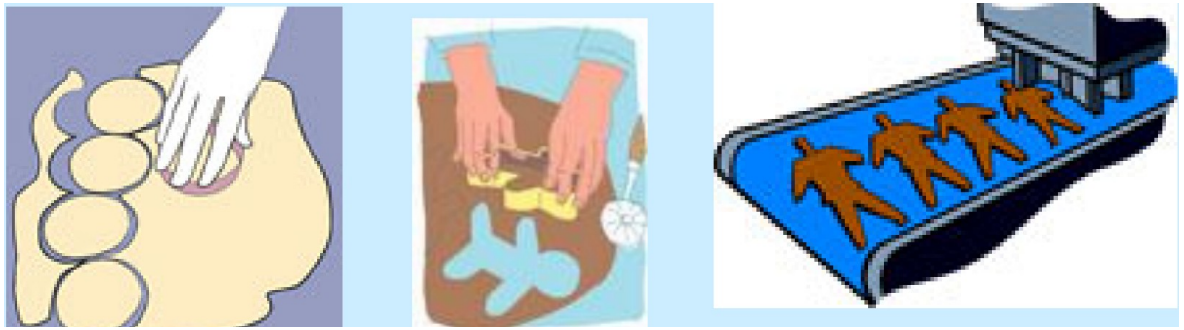
- **Software engineering (SE)** is the discipline of managing code to ensure its long-term use.
  - takes existing code and modifies it.
  - makes the overall code simpler, easier to understand.
  - doesn't change the functionality, only the form!

What is a Class?

- ***A class*** is a template for creating objects in Python. It defines the structure and behavior of objects by specifying attributes and methods.
- Classes allow you to create **instances** (objects) that share the same attributes and methods defined by the class. This concept enables reusability and organization of code.
- ***Example:*** Imagine a "Student" class with attributes like *name* and *age*, and methods like "*study*" and "*take_exam*." Multiple student instances can be created based on this class.

Class versus Instance

- Class: Defines attributes and methods that instances will have.
- Instance: Represents a specific object with its unique data.
- Example: The analogy of the cookie cutter and a cookie.



Class versus Instance

- In OOP, You define a class as a way to generate new instances of that class.
- Both the instances and the classes are themselves objects.
- The structure of an instance starts out the same, as dictated by the class.
- The instances respond to the messages defined as part of the class.

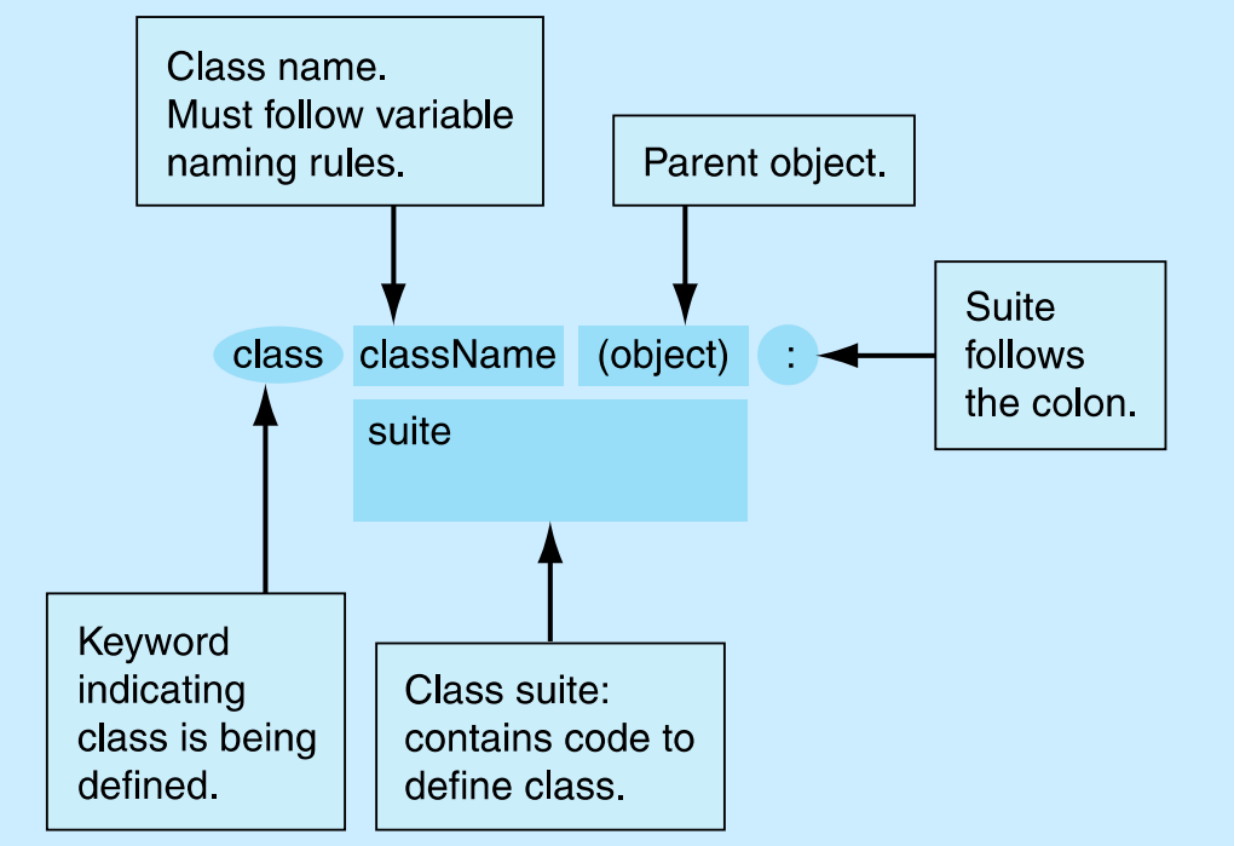
Why a class

- We make classes because we need more complicated, user-defined data types to construct instances we can use.
- Each class has potentially two aspects:
  - **Attributes**: Properties that store data related to the class. The data (types, number, names) that each instance might contain.
  - **Methods**: Functions defined within the class that perform actions. The messages that each instance can respond to.

Anatomy of a Class

- **Class Name:** The name used to define the class.
  - The standard way to name a class in Python is called ***CapWords***:
    - Each word of a class begins with a Capital letter.
    - no underlines.
    - sometimes called ***CamelCase***.
    - makes recognizing a class easier.
- **Attributes:** Properties that store data related to the class.
- **Methods:** Functions defined within the class that perform actions.
- **Constructor (`__init__` method):** A special method that initializes attributes when an instance is created.

The basic format of a class definition



Creating a Class in Python

```
class ClassName:
    def __init__(self, parameters):
    # Initialize attributes

    def method_name(self, parameters):
    # Define methods
```

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def start_engine(self):
        print(f"The {self.brand} {self.model}'s engine is running.")
```

dir() function

- The **dir()** function lists all the attributes of a class
- You can think of these as keys in a dictionary stored in the class.

pass keyword

The **pass** keyword is used to signify that you have intentionally left some part of a definition (of a function, of a class) undefined

- by making the suite of a class undefined, we get only those things that Python defines for us automatically.

```
>>> class MyClass (object):
        pass

>>> dir(MyClass)
'__class__', '__delattr__', '__dict__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattribute__', '__gt__', '__hash__',
'__init__', '__le__', '__lt__', '__module__', '__ne__', '__new__',
'__reduce__', '__reduce_ex__', '__repr__', '__setattr__',
'__sizeof__', '__str__', '__subclasshook__', '__weakref__']

>>> my_instance = MyClass()
>>> dir(my_instance)
['__class__', '__delattr__', '__dict__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattribute__', '__gt__', '__hash__',
'__init__', '__le__', '__lt__', '__module__', '__ne__', '__new__',
'__reduce__', '__reduce_ex__', '__repr__', '__setattr__',
'__sizeof__', '__str__', '__subclasshook__', '__weakref__']

>>> type(my_instance)
<class '__main__.MyClass'>
```

Constructor

- When a class is defined, a function is made with the same name as the class.
- This function is called the **constructor**. By calling it, you can create an instance of the class.
- **Constructor** (`__init__ method`): A special method that initializes attributes when an instance is created.
- We can affect this creation (more later), but by default Python can make an instance.

Dot reference

- We can refer to the attributes of an object by doing a dot reference, of the form:

```
object.attribute
```

- The attribute can be a variable or a function.
- It is part of the object, either directly or by that object being part of a class.

dot reference

- Examples,

```
print(my_instance.my_val)
```

Print a variable associated with the object `my_instance`

```
my_instance.my_method()
```

Call a method associated with the object `my_instance`.

- *Variable* versus *method*, you can tell by the parenthesis at the end of the reference.

An object-local value

- Once an object is made, the data is made by the same way as in any other Python situation, by assignment
- Any object can thus be augmented by adding a variable

```
my_instance.attribute = 'hello'
```

- New attribute shown in **dir**

```
dir(my_instance)
  - ['__class__', '__delattr__', '__dict__',
    '__doc__', '__format__', '__getattribute__',
    '__hash__', '__init__', '__module__',
    '__new__', '__reduce__', '__reduce_ex__',
    '__repr__', '__setattr__', '__sizeof__',
    '__str__', '__subclasshook__', '__weakref__',
    attribute]
```

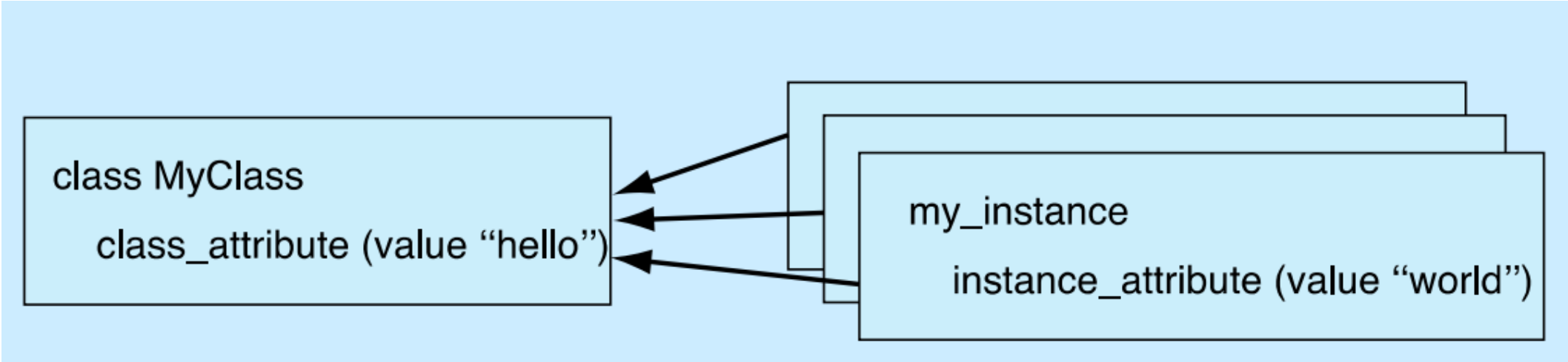
Class instance relationship

- Because each instance has its type of the class that it was made from, an instance remembers its class.
- This is often called the **instance-of** relationship
- Stored in the `__class__` attribute of the instance

```
>>> class MyClass (object):
        pass

>>> my_instance = MyClass()
>>> MyClass.class_attribute = 'hello'
>>> my_instance.instance_attribute = 'world'
>>> dir(my_instance)
['__class__', ... , 'class_attribute', 'instance_attribute']
>>> print(my_instance.__class__)
<class '__main__.MyClass'>
>>> type(my_instance)
<class '__main__.MyClass'>
>>> print(my_instance.instance_attribute)
world
>>> print(my_instance.class_attribute)
hello
>>> print(MyClass.instance_attribute)

Traceback (most recent call last):
  File "<pyshell#11>", line 1, in <module>
    print MyClass.instance_attribute
AttributeError: type object 'MyClass' has
no attribute 'instance_attribute'
```

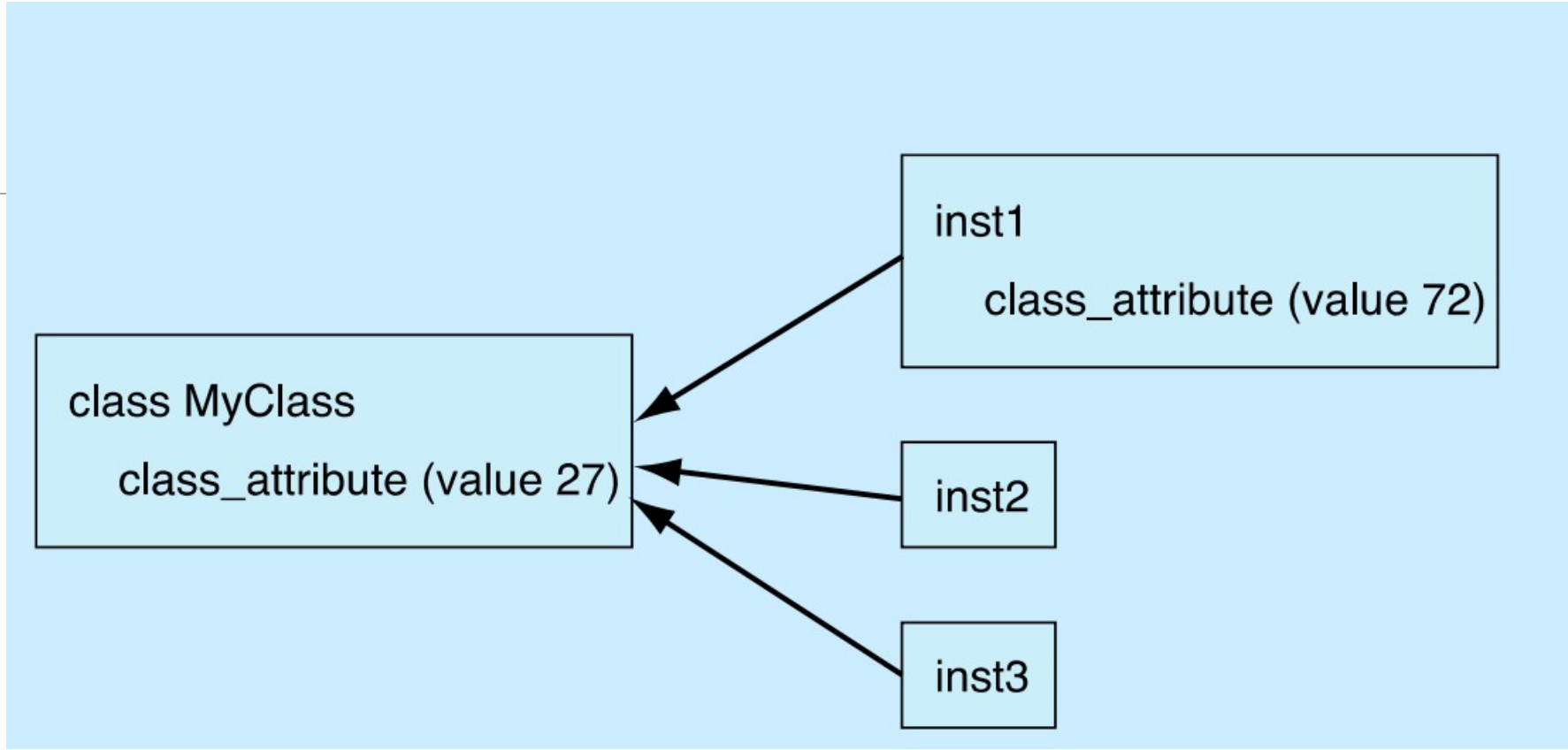

class MyClass
class_attribute (value “hello”)
my_instance
instance_attribute (value “world”)

Scope

- **Scope** refers to the region in a program where a variable can be accessed or modified.
- The **Object Scope Rule** is an important aspect of variable access within classes and objects in Python.
- The **Object Scope Rule** involves three main components:
  1. *Class Attributes* -> defined within the class itself.
  2. *Instance Attributes* -> specific to each instance.
  3. Methods -> functions defined within the class.

```
>>> class MyClass (object):
        pass

>>> inst1 = MyClass()
>>> inst2 = MyClass()
>>> inst3 = MyClass()
>>> MyClass.class_attribute = 27
>>> inst1.class_attribute = 72
>>> print(inst1.class_attribute)
72
>>> print(inst2.class_attribute)
27
>>> print(inst3.class_attribute)
27
>>> MyClass.class_attribute = 999
>>> print(inst1.class_attribute)
72
>>> print(inst2.class_attribute)
999
>>> print(inst3.class_attribute)
999
```


inst1
class_attribute (value 72)
class MyClass
class_attribute (value 27)
inst2
inst3

Method

```
class MyClass (object):
    class_attribute = 'world'

    def my_method (self, param1):
        print('\nhello {}'.format(param1))
        print('The object that called this method is: {}'.\
              format(str(self)))
        self.instance_attribute = param1


my_instance = MyClass()
print("output of dir(my_instance):")
print(dir(my_instance))
my_instance.my_method('world')              # adds the instance_attribute
print("Instance has new attribute with value: {}".\
      format(my_instance.instance_attribute))
print("output of dir(my_instance):")
print(dir(my_instance))
```

Method versus Function

- A method and a function are closely related.
- They are both “small programs” that have parameters, perform some operation and (potentially) return a value.
- The main difference is that methods are functions tied to a particular object.

Difference in calling

- Function:

```
do_something(param1)
```

- Method:

```
an_object.do_something(param1)
```

This means that the object that the method is called on is always ***implicitly*** a parameter!

Difference in definition

- Methods are defined ***inside*** the suite of a class.
- Methods always bind the first parameter in the definition to the object that called it.
- This parameter can be named anything, but traditionally it is named ***self***

```
class MyClass:
  def my_method(self,param1):
 suite
```

more on *self*

- ***self*** is an important variable. In any method it is bound to the object that called the method.
- Through ***self***, we can access the instance that called the method (and all of its attributes as a result)

```
class MyClass (object):
    class_attribute = 'world'

    def my_method (self, param1):
        print('\nhello {}'.format(param1))
        print('The object that called this method is: {}'.\
              format(str(self)))
        self.instance_attribute = param1
```

Binding *self*

```
my_instance = MyClass()
my_instance.my_method( "world" )

class MyClass (object):
        def my_method (self, param1):
            #method suite
```

Binding *self*

- When a dot method call is made, the object that called the method is **automatically** assigned to ***self*.**
- We can use ***self*** to remember, and therefore refer, to the calling object
- to reference any part of the calling object, we must always precede it with ***self***.
- The method can be written generically, dealing with calling objects through ***self***

End of the Lecture